Part 3 — Terraform Deep Dive

Probability: 🟠 85% — Very Frequently Asked

Core Concepts

Provider Configuration

```
# providers.tf
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"            # Pessimistic constraint: >=3.0, <4.0
    }
  }

  # Remote backend for team collaboration
  backend "azurerm" {
    resource_group_name  = "rg-terraform-state"
    storage_account_name = "tfstateinfosys"
    container_name       = "tfstate"
    key                  = "prod.terraform.tfstate"
  }
}

provider "azurerm" {
  features {}                        # Required block, even if empty
  subscription_id = var.subscription_id
}
```

Q: "What does `~> 3.0` mean?"

> *"It's a pessimistic version constraint — allows 3.x.x but not 4.0.0. Protects against breaking changes while allowing patches. Similar to `^3.0.0` in npm."*

Variables

```
# variables.tf
variable "environment" {
  type        = string
  description = "Environment name (dev, staging, prod)"
  default     = "dev"
```

```
  validation {                        # Input validation
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Environment must be dev, staging, or prod."
  }
}

variable "vm_size" {
  type    = string
  default = "Standard_B2s"
}

variable "tags" {
  type = map(string)
  default = {
    managed_by = "terraform"
    team       = "devops"
  }
}

variable "db_password" {
  type      = string
  sensitive = true                    # Masked in output and logs
}
```

Q: "How do you manage sensitive variables?"

> *"Mark them `sensitive = true` — Terraform masks them in plan/apply output. Never store them in `.tfvars` files committed to Git. Options: 1) Environment variables (`TF_VAR_db_password`), 2) Azure Key Vault data source, 3) Terraform Cloud variables marked sensitive, 4) CI/CD pipeline secret variables injected at runtime."*

Outputs & Locals

```
# outputs.tf
output "aks_cluster_name" {
  value       = azurerm_kubernetes_cluster.main.name
  description = "Name of the AKS cluster"
}

output "db_connection_string" {
  value     = azurerm_postgresql_server.main.fqdn
  sensitive = true                    # Sensitive output
}

# locals.tf
locals {
  name_prefix = "${var.project}-${var.environment}"
  common_tags = merge(var.tags, {
    environment = var.environment
    project     = var.project
```

```
  })
}
```

Azure Resource Examples

Resource Group + VNet + Subnet + NSG

```
resource "azurerm_resource_group" "main" {
  name     = "${local.name_prefix}-rg"
  location = var.location
  tags     = local.common_tags
}

resource "azurerm_virtual_network" "main" {
  name                = "${local.name_prefix}-vnet"
  resource_group_name = azurerm_resource_group.main.name
  location            = azurerm_resource_group.main.location
  address_space       = ["10.0.0.0/16"]
  tags                = local.common_tags
}

resource "azurerm_subnet" "app" {
  name                 = "app-subnet"
  resource_group_name  = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.0.1.0/24"]
}

resource "azurerm_network_security_group" "app" {
  name                = "${local.name_prefix}-app-nsg"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name

  security_rule {
    name                       = "AllowHTTPS"
    priority                   = 100
    direction                  = "Inbound"
    access                     = "Allow"
    protocol                   = "Tcp"
    source_port_range          = "*"
    destination_port_range     = "443"
    source_address_prefix      = "*"
    destination_address_prefix = "*"
  }

  security_rule {
    name                       = "DenyAll"
    priority                   = 4096
    direction                  = "Inbound"
    access                     = "Deny"
    protocol                   = "*"
    source_port_range          = "*"
    destination_port_range     = "*"
```

```
    source_address_prefix      = "*"
    destination_address_prefix = "*"
  }
}
```

AKS Cluster

```
resource "azurerm_kubernetes_cluster" "main" {
  name                = "${local.name_prefix}-aks"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  dns_prefix          = local.name_prefix
  kubernetes_version  = "1.28"

  default_node_pool {
    name                = "system"
    node_count          = 3
    vm_size             = "Standard_D4s_v3"
    vnet_subnet_id      = azurerm_subnet.app.id
    enable_auto_scaling = true
    min_count           = 2
    max_count           = 5
    os_disk_size_gb     = 50
  }

  identity {
    type = "SystemAssigned"          # Managed Identity — no secrets to manage
  }

  network_profile {
    network_plugin = "azure"         # Azure CNI for VNet integration
    service_cidr   = "10.1.0.0/16"
    dns_service_ip = "10.1.0.10"
  }

  tags = local.common_tags
}
```

Modules

```
# modules/aks/main.tf — Reusable module
variable "cluster_name" { type = string }
variable "node_count"   { type = number; default = 3 }
variable "vm_size"      { type = string; default = "Standard_D4s_v3" }

resource "azurerm_kubernetes_cluster" "this" {
  name                = var.cluster_name
  # ... (full config)
}

output "kube_config" {
  value     = azurerm_kubernetes_cluster.this.kube_config_raw
  sensitive = true
}
```

```
# Root module usage
module "aks_prod" {
  source       = "./modules/aks"     # Local module
  # source     = "git::https://github.com/org/tf-modules.git//aks?ref=v2.1.0"  # Ve
  cluster_name = "prod-aks"
  node_count   = 5
  vm_size      = "Standard_D8s_v3"
}
```

Advanced Concepts

count vs for_each

```
# count — use for identical resources
resource "azurerm_subnet" "workers" {
  count                = 3
  name                 = "worker-subnet-${count.index}"
  address_prefixes     = ["10.0.${count.index + 10}.0/24"]
  resource_group_name  = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name
}

# for_each — use for distinct resources (PREFERRED)
variable "environments" {
  default = {
    dev     = { size = "Standard_B2s", count = 1 }
    staging = { size = "Standard_D2s_v3", count = 2 }
    prod    = { size = "Standard_D4s_v3", count = 3 }
  }
}

resource "azurerm_kubernetes_cluster" "env" {
  for_each            = var.environments
  name                = "${each.key}-aks"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  dns_prefix          = each.key

  default_node_pool {
    name       = "default"
    node_count = each.value.count
    vm_size    = each.value.size
  }
  identity { type = "SystemAssigned" }
}
```

Q: "When do you use count vs for_each?"

> *"Use `for_each` when resources are distinct (different configs, names). Use `count` only for truly identical resources. Key difference: `count` uses index — if you remove item at index 1, all subsequent*

> *items shift and get recreated. `for_each` uses map keys — removing one item doesn't affect others. `for_each` is almost always the safer choice."*

Lifecycle & depends_on

```
resource "azurerm_kubernetes_cluster" "main" {
  # ...

  lifecycle {
    create_before_destroy = true     # Zero-downtime replacement
    prevent_destroy       = true     # Prevent accidental deletion
    ignore_changes        = [tags]   # Ignore external tag changes
  }
}

# Explicit dependency (when Terraform can't infer)
resource "azurerm_role_assignment" "aks_acr" {
  principal_id         = azurerm_kubernetes_cluster.main.kubelet_identity[0].object
  role_definition_name = "AcrPull"
  scope                = azurerm_container_registry.main.id

  depends_on = [azurerm_kubernetes_cluster.main]
}
```

State Management

Q: "What happens if two people run `terraform apply` at the same time?"

> *"Without state locking, you get state corruption. That's why we use remote backends with locking: Azure Blob Storage uses blob leasing for locks (no DynamoDB needed like AWS). When one person runs apply, the state file is locked. Others see 'Error: Error locking state' and must wait. If a lock is stuck due to a crashed process, you can use `terraform force-unlock <LOCK_ID>` — but ONLY after confirming no apply is actually running."*

Q: "How do you recover from a corrupted state file?"

> *"1) If using remote backend with versioning enabled — restore previous version from blob storage. 2) If no backup — use `terraform import` to reconstruct state from existing infrastructure, resource by resource. 3) Prevention: always enable versioning on your state storage, run regular state backups, and use CI/CD for applies (never local)."*

Workspaces

```
terraform workspace new dev
terraform workspace new staging
```

```
terraform workspace new prod
terraform workspace select prod
terraform workspace list
```

Q: "Workspaces vs separate directories for environments?"

> *"I prefer separate directories (or separate root modules) with a shared module. Workspaces share the same code and backend — you can accidentally apply dev config to prod if you forget which workspace you're in. Separate directories with remote state make the blast radius explicit. Workspaces are fine for ephemeral environments (PR previews, testing)."*

Commands Cheat Sheet

Command	Purpose	When to Use
`terraform init`	Initialize, download providers/modules	First time, after adding providers
`terraform plan`	Preview changes	ALWAYS before apply
`terraform apply`	Execute changes	After reviewing plan
`terraform destroy`	Delete all resources	Teardown environments
`terraform fmt`	Format code	Before every commit
`terraform validate`	Syntax/logic validation	In CI pipeline
`terraform import`	Import existing resources	Brownfield adoption
`terraform state list`	List tracked resources	Debugging
`terraform state show`	Show resource details	Debugging
`terraform state mv`	Move/rename in state	Refactoring
`terraform state rm`	Remove from state (not infra)	Unmanage a resource
`terraform output`	Show outputs	Get values for scripts
`terraform taint` (deprecated)	Mark for recreation	Use `-replace` flag instead
`terraform plan -replace=RES`	Force recreate specific resource	Replace unhealthy resource

---

Terraform Troubleshooting — 50 Scenarios

#	Scenario	Root Cause	Fix
1	`Error: Error locking state`	Another apply is running or lock stuck	Wait, or `terraform force-unlock <ID>` after verification
2	State file corrupted	Concurrent apply without locking	Restore from versioned backend backup
3	`Error: Provider not found`	Missing `terraform init`	Run `terraform init`
4	`Error: Unsupported attribute`	Wrong provider version	Check docs, pin correct version
5	Resource recreated unexpectedly	Change in an immutable attribute (e.g., name)	Use `lifecycle { create_before_destroy = true }`
6	Drift detected in plan	Manual changes in console	`terraform apply` to reconcile, or update code to match
7	`Error: Cycle detected`	Circular dependency	Refactor — break cycle with intermediate resources or data sources
8	Plan shows destroying resources after `count` change	Index shift	Migrate to `for_each`
9	`terraform import` fails	Wrong resource address or ID format	Check provider docs for exact import ID format
10	Module source changed but not updating	Cached locally	`terraform init -upgrade`
11	Sensitive value shown in plan	Missing `sensitive = true`	Add `sensitive = true` to variable/output
12	`Error: creating X: StatusCode=403`	Insufficient permissions on Service Principal	Check RBAC, assign required roles
13	`Error: A resource with the ID already exists`	Resource exists but not in state	`terraform import` to bring into state
14	Apply times out	Long-running resource creation (AKS ~10-15 min)	Increase timeout in resource block
15	`Error: Invalid count argument`	Count depends on computed value	Use `depends_on` or restructure
16	State too large, slow plans	Monolithic state file	Split into smaller state files by component
17	`Error: Backend configuration changed`	Backend block modified	`terraform init -reconfigure` or `-migrate-state`
18	Destroy hangs on a resource	Resource has dependencies in cloud	Delete manually, then `terraform state rm`

#	Scenario	Root Cause	Fix
19	`Error: Module not installed`	Missing init after adding module	`terraform init`
20	Variable validation fails	Input doesn't match constraint	Check validation block conditions
21	Output shows `<sensitive>`	Sensitive flag set	Use `terraform output -json` to see values
22	`Error: creating NSG Rule: Rule with priority X exists`	Duplicate priority	Use unique priority numbers per NSG
23	Subnet conflicts with existing CIDR	Address space overlap	Choose non-overlapping CIDR ranges
24	AKS node pool won't scale	Quota limit reached	Request quota increase in Azure portal
25	`moved` block not working	Wrong `from` address	Use `terraform state list` to verify exact address
26	Provisioner fails	Network/SSH issue to target	Use `null_resource` with proper connection block
27	Remote exec timeout	SSH not reachable	Check NSG rules, public IP, SSH key
28	State shows resource but it's deleted manually	Orphaned state entry	`terraform state rm` the orphan, then `terraform apply`
29	Plan shows no changes but infra is wrong	State and infra in sync but code is wrong	`terraform refresh` (now part of plan), review carefully
30	`for_each` with list fails	`for_each` requires map or set	Use `toset()` or convert list to map
31	Dynamic block generates wrong config	Incorrect iterator	Check `content` block references
32	Terraform Cloud run queue stuck	Previous run pending	Discard pending run in TFC UI
33	`Error: Provider produced inconsistent result`	Provider bug or API race condition	Re-run apply, or report to provider
34	Can't delete resource group (has locks)	Management lock exists	Remove lock first, then destroy
35	Key Vault soft-delete conflict	Deleted vault with same name exists	Purge deleted vault first
36	AKS upgrade fails mid-way	Node pool version incompatibility	Can't skip versions — upgrade incrementally
37	`Error: retrieving X: unexpected status 404`	Resource deleted outside Terraform	`terraform state rm`, then re-apply
38	Large number of API calls hitting rate limits	Too many resources in one apply	Split state, use `-parallelism=5` flag

#	Scenario	Root Cause	Fix
39	Conditional resource creation unclear	Using `count = var.enabled ? 1 : 0`	Standard pattern — document clearly
40	Module version pinning not working	Using branch ref instead of tag	Use `?ref=v1.2.3` (tags, not branches)
41	`terraform plan` takes 10+ minutes	Large state, many resources	Split state, use targeted plans: `-target`
42	Accidental `terraform destroy` on prod	No safety net	Use `prevent_destroy = true` on critical resources
43	Environment variable not picked up	Wrong naming	Must be `TF_VAR_<name>`, case-sensitive
44	Data source returns empty	Filter doesn't match any resource	Verify filters, check resource exists
45	`Error: Multiple configurations for provider`	Duplicate provider blocks	Remove duplicate, use aliases for multi-region
46	Null values in `for_each` map	Map contains null values	Filter nulls: `{ for k, v in var.map : k => v if v != null }`
47	`terraform plan -out=plan.out` corrupted	Binary file handling issue	Regenerate plan file
48	Workspace-specific variable not applied	Wrong workspace selected	`terraform workspace show` to verify
49	`Error: error configuring S3 Backend`	AWS credentials for backend expired	Refresh credentials, re-init
50	Import then plan shows changes	Imported state doesn't match code exactly	Adjust code to match imported state, or accept the diff